Node-ID / Offset Example Management and Histogram Build

Oct 23, 2026 · @Jung Yoon

Overview

The node-id path never physically moves example indices between depths. Each dataset row carries a node id; each depth derives a per-node rank (the offset) from those ids in one pass. Example management then costs one read of `d_node_ids` per depth plus a rewrite of that array at split time.

The baseline path keeps `d_selected_examples` sorted by node and rebuilds it every depth with a partition (`SplitExamplesInPlaceKernel_all_depth_No_D2H_fused`). That rebuild, plus the scattered label gathers it forces on the histogram, is what the node-id path removes.

Each BFS depth on the node-id path runs four stages:

1. **Compute offsets** — `NodeIdDRSW_ComputeOffsetsOnly`: from `d_node_ids`, give every row its rank within its node, and scatter its label (and dup count) into node-sorted order.
2. **Apply projection** — `NodeIdDRSW_ProjectionOnly` (row-ordered DRSW) or, under `nid_srdw_proj`, `ColumnAddProjectionKernel_SRDW_1_2D` (node-sorted gather).
3. **Histogram** — `BuildHistogramSRDWLabelsKernel`: one block per node, coalesced reads of projected values and the pre-scattered labels.
4. **Split update** — `NodeIdDRSW_SplitUpdate`: each row reads its node's best projection and threshold and writes its child id back into `d_node_ids`.

All numbers in this doc come from HIGGS1 (11,000,000 rows × 28 features) on an L4 GPU, seed 1, sampling without replacement at `bootstrap_size_ratio=1.0`, density factor 1.5.

Data structures

Two arrays indexed by dataset row (`d_node_ids`, `d_node_offsets`) define the whole partition; everything node-sorted is derived from them each depth.

Array	Size	Indexed by	Holds	Written by
`d_node_ids`	`num_rows`	dataset row	node id at this depth, `-1` = not in the bag or in a finished leaf	Init once per tree; SplitUpdate each depth
`d_node_offsets`	`num_rows`	dataset row	rank of the row within its node (`local_r`)	ComputeOffsetsOnly
`d_node_row_off`	`num_nodes + 1`	node	prefix sum of node sizes: node `n` owns slots `[off[n], off[n+1])`	host, from SplitUpdate child counts
`d_labels_scattered`	`num_rows`	slot	class label (1 or 2) of the row in that slot	ComputeOffsetsOnly (`WRITE_LABELS`)
`d_dup_scattered`	`num_rows`	slot	bootstrap multiplicity of that row (`DEDUP_BOOTSTRAP` only)	ComputeOffsetsOnly (`WRITE_LABELS`)
`d_selected_examples`	`total_rows`	slot	dataset row in that slot	Init input; ComputeOffsetsOnly only under `nid_srdw_proj`
`d_col_add_projected`	`total_rows × num_proj`	node, proj, slot	projected value, at `row_start·num_proj + proj·rows_node + local_r`	projection kernel

A **slot** is a position in node-sorted order: `slot = d_node_row_off[nid] + d_node_offsets[row]` . Labels and dup counts are values; only `d_selected_examples` stores indices (row ids).

Worked example: 10 rows, 2 nodes

Node 0 holds rows {0, 3, 5, 6, 9} and node 1 holds rows {1, 2, 4, 7, 8}, so `d_node_row_off = [0, 5, 10]` .

row	0	1	2	3	4	5	6	7	8	9
`d_node_ids[row]`	0	1	1	0	1	0	0	1	1	0
`d_node_offsets[row]`	0	0	1	1	2	2	3	3	4	4

row	0	1	2	3	4	5	6	7	8	9
slot	0	5	6	1	7	2	3	8	9	4
`d_labels[row]`	1	2	2	1	1	2	1	2	1	2

Scattering by slot gives the node-sorted arrays:

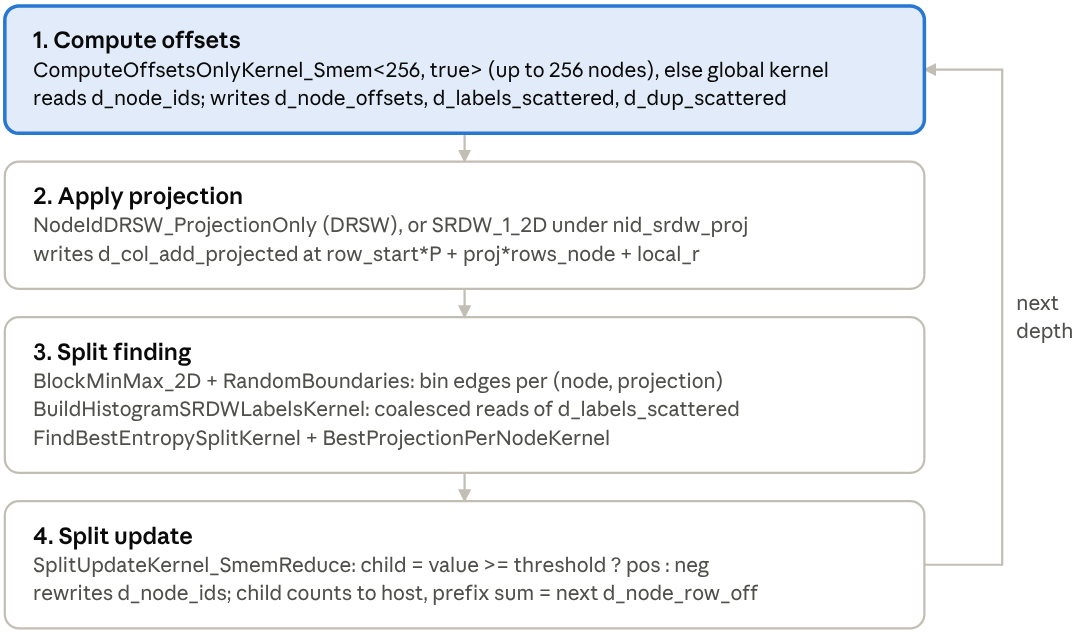
slot	0	1	2	3	4	5	6	7	8	9
`d_selected_examples[slot]`	0	3	5	6	9	1	2	4	7	8
`d_labels_scattered[slot]`	1	1	2	1	2	2	2	1	2	1

The ranks here are in ascending row order for readability. On the GPU, rank order within a node follows atomic arrival, so it is only roughly ascending; every consumer reads through the same offsets, so any order is correct.

Per-depth pipeline

Only split update writes `d_node_ids`; every other stage reads it or reads arrays derived from it at the start of the depth.

Node ids drive every stage; only split update writes them



per-depth pipeline · node-id path in training.cc

The accented box is the example-management step this doc is about. Init runs once per tree before depth 1 and is not drawn.

Init (once per tree)

`NodeIdDRSW_InitKernel` runs one thread per sampled slot. It finds the slot's node with `cub::UpperBound` over `d_node_row_off` and writes `d_node_ids[d_selected_examples[pos]] = node_id`. Rows outside the bag stay `-1` from the preceding `cudaMemset`.

Compute offsets

`NodeIdDRSW_ComputeOffsetsOnly` launches one thread per dataset row and picks a kernel by node count:

- **256 nodes or fewer** — `NodeIdDRSW_ComputeOffsetsOnlyKernel_Smem<256, WRITE_LABELS>`. Each thread does `atomicAdd(&s_counts[nid], 1)` in shared memory to get its rank in the block. Thread `n` then claims a base for node `n` with one `atomicAdd(&d_atomic_counters[n], s_counts[n])`. Offset = base + block rank.
- **More than 256 nodes** — `NodeIdDRSW_ComputeOffsetsOnlyKernel`. `__match_any_sync` groups lanes of a warp with the same node; the leader does one global `atomicAdd` for the group and lanes rank by `__popc` of lower lanes.

Both write `d_node_offsets[row]` and, with `WRITE_LABELS` (the `scattered_label_hist` build), `d_labels_scattered[slot]` and `d_dup_scattered[slot]`.

Split update

`NodeIdDRSW_SplitUpdateKernel_SmemReduce` is one thread per row. It reads the row's projected value for its node's best projection at `row_start·num_proj + best_proj·rows_node + d_node_offsets[row]`, compares it with the threshold and writes the child id into `d_node_ids[row]`. Leaves get `-1`. Child row counts and per-class counts accumulate in shared memory and flush with one global `atomicAdd` per key per block; the host turns the child counts into the next depth's `d_node_row_off`.

Histogram build

The node-id histogram reads everything in slot order, so every global load is coalesced; the baseline gathers each row's label by dataset index once per projection.

`BuildHistogramSRDWLabelsKernel` launches a 2D grid: `blockIdx.x` walks the blocks of all nodes (node found by `cub::UpperBound` over `d_node_block_off`), `blockIdx.y` is the projection. Per block:

1. Zero a shared histogram of `2 × num_bins` ints and load the segment's `num_bins − 1` candidate splits into shared memory.
2. Each thread takes local row `r` and reads `projected[row_start·num_proj + proj·rows_node + r]` and `d_labels_scattered[row_start + r]`.
3. `bin = cub::LowerBound(sh_splits, num_bins − 1, val)`; `atomicAdd(&sh_hist[(2 − label)·num_bins + bin], w)` with `w = d_dup_scattered[row_start + r]` under `DEDUP_BOOTSTRAP`, else 1.
4. Flush: thread `b` adds bin `b` of both classes into `d_hist_class0/1[seg·num_bins + b]` with one global `atomicAdd` each.

Global reads per row, per projection

Read	Baseline `BuildHistogramRandomKernel_2D`	Node-id `BuildHistogramSRDWLabelsKernel`
projected value	coalesced (slot order)	coalesced (slot order)
row id	`d_selected_examples[slot]`, coalesced	none
label	`d_labels[example_idx]`, scattered gather	`d_labels_scattered[slot]`, coalesced
dup count (dedup)	`d_ex_dup_counts[example_idx]`, scattered gather	`d_dup_scattered[slot]`, coalesced

The scatter that makes the right-hand column coalesced happens once per depth in compute offsets, not once per projection.

Measured histogram time

HIGGS1, depth 10, sum of `build_hist` over depths (from the October 13 run):

Histogram variant	build_hist	vs scattered labels
`BuildHistogramSRDWLabelsKernel` (scattered labels)	17.4 ms	1.0×
`BuildHistogramDRSWKernel` (node id + offset, row order)	48.2 ms	2.8×
`BuildHistogramRandomKernel_2D` (baseline gather)	84.8 ms	4.9×

The scattered-label kernel stays near 1.9 ms per depth; the baseline grows from about 3 ms to 15 ms as nodes shrink and the label gather spreads over more cache lines.

Putting selected examples into compute offsets

One extra store in the offsets kernel, `d_selected_examples[slot] = row`, rebuilds the node-sorted row list for free; it adds about 0.19 ms per depth at 11M rows (`--config=nid_srdw_proj`).

Why it is needed

`ColumnAddProjectionKernel_SRDW_1_2D` walks slots, not rows: thread `r` of node `n` needs the dataset row in slot `row_start + r` to read `dataset[col·num_rows + ex_idx]`. Plain `offsets_only` never rebuilds that list, so the SRDW kernel had no valid input on the node-id path.

The change

The thread that ranks a row already knows both the row and its slot, so it writes the row id at that slot — the same position as the label scatter, with no new atomics:

```
// ComputeOffsetsOnlyKernel_Smem tail, simplified (global kernel same)
const int slot = node_row_off[nid] + offset;
node_offsets[row] = offset;
d_labels_scattered[slot] = d_labels[row];   // WRITE_LABELS
d_dup_scattered[slot]    = d_dup[row];      // WRITE_LABELS + DEDUP_BOOTSTRAP
#ifdef NODE_ID_SRDW_PROJ
d_selected_examples[slot] = row;            // new: row list for SRDW_1_2D
#endif
```

The pointer travels as an optional last argument through `NodeIdDRSW_ComputeOffsetsOnly`, `NodeIdDRSW_ComputeOffsetsOnly_wrap` and the call in `training.cc`; projection then calls `ColumnAddProjectionKernel_SRDW_1_2D_wrap` instead of `NodeIdDRSW_ProjectionOnly_wrap`. Split update and histogram are unchanged, and the projected layout is identical, so the trees match.

Cost

The store adds 11M × 4 B = 44 MB of writes per depth; at the L4's ~300 GB/s that is about 0.15 ms, close to the measured gap below. It is a pure write to the slot the label scatter already computes.

Measured results

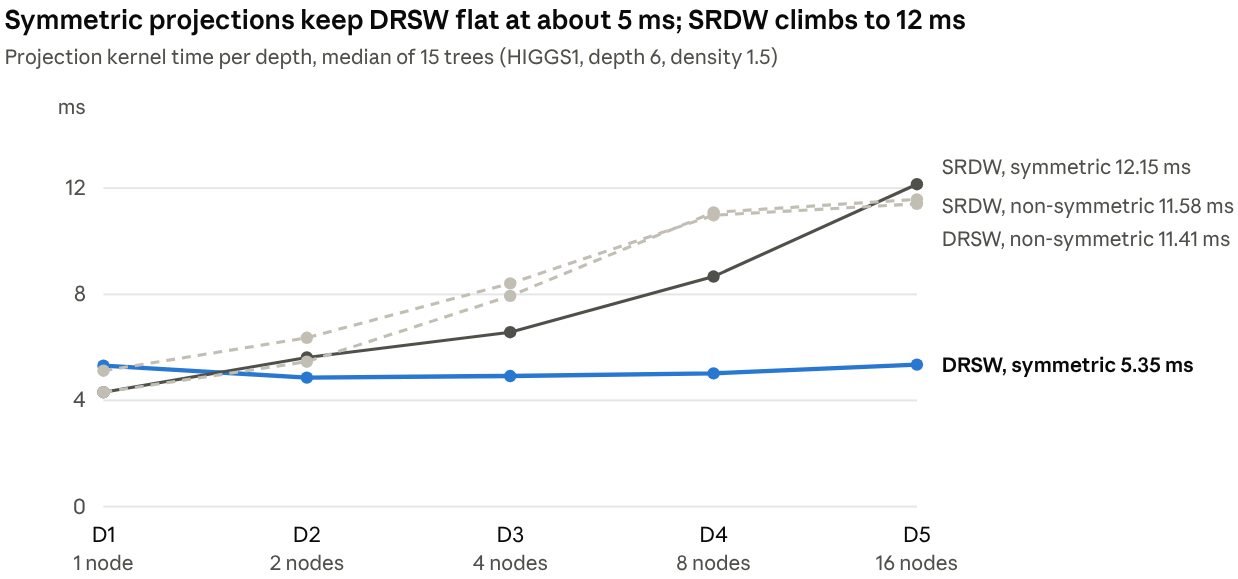
Adding the row-id store costs a flat 0.19 ms per depth; whether SRDW or DRSW projection wins then depends on `symmetric_proj`. All runs: HIGGS1, depth 6, 5 trees × 3 alternating repeats (offsets table: 2 trees, one run).

Compute offsets per depth

Depth	Nodes	Offsets only (ms)	Offsets + row ids, `nid_srdw_proj` (ms)	Extra (ms)
D1	1	1.57	1.75	+0.18
D2	2	0.99	1.18	+0.19
D3	4	0.99	1.18	+0.19
D4	8	0.99	1.18	+0.19
D5	16	0.99	1.19	+0.20
Total, 2 trees		**5.54**	**6.47**	**+0.94**

Both builds use `NodeIdDRSW_ComputeOffsetsOnlyKernel_Smem<256, true>` at every depth here (at most 16 nodes).

Projection kernel per depth



HIGGS1 benchmark runs, 2026-10-23 · 3 alternating repeats × 5 trees per build

Without symmetric projections the two kernels tie: SRDW's gather is near-sequential while nodes are few, and DRSW warps mix nodes that read different columns. With `symmetric_proj` every node uses the same columns for a projection, so DRSW warps read one column coalesced and stay flat; SRDW gains nothing because its blocks are already per node.

Per tree

Build	Projection kernel	Tree total
DRSW, symmetric	25.1 ms	65.3 ms
SRDW (`nid_srdw_proj`), symmetric	37.9 ms	78.9 ms
DRSW, non-symmetric	40.6 ms	80.6 ms
SRDW (`nid_srdw_proj`), non-symmetric	39.0 ms	79.9 ms

Repeats agreed within 0.1 ms. Every pair produced identical per-depth node counts.

Build configs and source locations

Every variant here builds from `main` on top of the same base set:

```
--config=bfs_only --config=use_gpu --config=node_id_drsw \
--config=offsets_only --config=scattered_label_hist
```

`--config=`	Macro	Effect	Guard
(none)	—	DRSW `ProjectionOnly` projection	—
`nid_srdw_proj`	`NODE_ID_SRDW_PROJ`	offsets kernel also writes `d_selected_examples`; SRDW_1_2D projection	`#error` without `scattered_label_hist`
`symmetric_proj`	`SYMMETRIC_PROJ`	all nodes at a depth share projections	—

`offsets_only` requires `scattered_label_hist` (`#error` in `training.cc`), and `scattered_label_hist` requires both `node_id_drsw` and `offsets_only`.

Paths below are under `yggdrasil_decision_forests/learner/decision_tree/`.

File	Holds
`node_id_drsw.cuh`	Init, both ComputeOffsetsOnly kernels, SplitUpdate
`oblique_apply_projection.cu`	histogram and projection kernels, the `_wrap` functions
`training.cc`	per-depth sequence in `GrowTreeLocalBFS`, `#error` guards
`BUILD` (and `.bazelrc` at the repo root)	build flags